DOSSIER DE PROJET

DRIVEO

Location de voitures de luxe

Janice Kouakou
Holberton School
Stages 1 à 3 · Octobre 2026



Sommaire

Stage 1 **Équipe, idées et MVP**

Stage 2 **Planification du projet**

Stage 3 **Documentation technique**

Driveo est un site de location de voitures de luxe : catalogue, prix en direct, comptes utilisateurs et réservations sécurisées. Code : github.com/Janice91/metricflow · Site : metricflow-tan.vercel.app

STAGE 1

Équipe, idées et MVP

1. Team Formation Overview

Pour Driveo, je mène le projet en autonomie : je cumule les rôles techniques et la gestion de projet. Mes mentors Holberton et les futurs utilisateurs du site sont les parties prenantes qui cadrent et valident mon travail.

Membre et rôle principal

Membre	Rôle principal	Pourquoi ce rôle
Moi, Janice Kouakou	Project Manager temporaire et Fullstack Developer	J'ai de l'expérience en Python, JavaScript, Flask, MongoDB, Docker et API REST grâce aux projets HBnB, puis en Next.js et Supabase avec MetricFlow. Je peux coder le front et le back et coordonner l'ensemble.

Rôles techniques et responsabilités

Je distingue chaque rôle, même si je les tiens tous, pour que mon travail reste bien structuré et que je sache quelles décisions relèvent de quel rôle.

Rôle	Responsabilités	Outils
Project Manager temporaire	Planifier les étapes, suivre l'avancement, prendre les décisions, faire le lien avec les mentors	Liste de tâches, journal d'avancement
Frontend Developer	Pages (accueil, catalogue, fiche voiture, connexion, réservation, mes réservations), composants, responsive	Next.js 16, React 19, TypeScript, Tailwind CSS
Backend Developer	Comptes utilisateurs, base de données des réservations, règles de sécurité (RLS)	Supabase Auth, Postgres, supabase-js
UI Designer	Identité noir et or, choix des photos, cohérence visuelle	Tailwind CSS, photos Wikimedia Commons
DevOps	Versionner le code, mettre en ligne, gérer les variables d'environnement	Git, GitHub, Vercel CLI
QA / testeur	Tester le parcours complet (recherche, connexion, réservation, annulation) et corriger	Navigateur, tests manuels

Normes de communication et de décision

- **Suivi quotidien** : je commence chaque session par un point de 15 minutes pour fixer la tâche du jour, et je fais un bilan à la fin.
- **Code** : je versionne sur GitHub, avec un commit par fonctionnalité et un message clair.
- **Retours externes** : je pose mes questions aux mentors Holberton et j'intègre leur feedback à chaque étape.
- **Règle de décision** : je choisis selon quatre critères (faisabilité dans le temps imparti, utilité pour l'utilisateur, alignement avec les technologies apprises, évolutivité). En cas de doute, je demande l'avis d'un mentor.

Parties prenantes

Partie prenante	Rôle	Impact sur le projet
Mentors et équipe pédagogique Holberton	Évaluateurs	Valident chaque étape, donnent le feedback : impact fort
Utilisateurs finaux (clients souhaitant louer une voiture de luxe)	Utilisateurs cibles	Définissent les besoins : prix clair, réservation rapide, suivi : impact fort
Agences de location (clients à servir)	Commanditaires fictifs du site	Fixent les règles métier : âge minimum, remises, annulation : impact moyen
Supabase, Vercel, GitHub	Fournisseurs techniques	Hébergement, comptes, base de données, code : impact moyen
Auteurs des photos (Wikimedia Commons)	Fournisseurs d'images libres	Fournissent les photos, créditées sur le site : impact faible
Camarades de promotion	Pairs	Retours et revue du projet : impact faible

2. Ideas Explored (idées brainstormées)

J'ai utilisé le **mind mapping** et les questions **« How Might We »** pour explorer plusieurs pistes autour de problèmes concrets du quotidien et des petites entreprises.

#	Idée	Description courte	Raison du rejet ou du choix
1	App de gestion de tâches collaborative	Un Trello simplifié pour petites équipes	Marché déjà saturé (Trello, Notion, Asana), peu de valeur différenciante
2	App de suivi de dépenses personnelles	Suivi de budget avec visualisation	Trop simple techniquement, peu de défi côté back-end
3	Plateforme de mise en relation freelances	Matching freelances et clients	Scope trop large : modération, paiement, aspects légaux
4	App de réservation de salles de sport	Réservation de créneaux	Trop proche d'exemples déjà vus en formation
5	MetricFlow : dashboard pour agences de publicité	Suivi des performances publicitaires et rapports IA	Bonne idée, mais dépend d'API externes (Meta Ads) difficiles à tester et peu visuelle à présenter. J'en garde la base technique (Next.js, Supabase).
6	**Driveo : site de location de voitures de luxe**	Choisir une voiture, voir le prix total, réserver et suivre ses locations	**Retenue**

Critères d'évaluation utilisés

- **Faisabilité technique** dans le temps imparti et avec mes compétences actuelles.
- **Utilité réelle** pour un utilisateur concret.
- **Alignement avec les technologies apprises** (Next.js, API REST, bases de données).
- **Évolutivité** : le projet peut grandir sur les étapes suivantes.
- **Impact visuel** : un site que l'on peut montrer et démontrer en direct.

---

3. Selected MVP Concept — Driveo

Résumé

Driveo est un site web de **location de voitures de luxe**. Un client peut parcourir un catalogue de 12 voitures (Urus Performante, Corvette C8, Ferrari 296 GTB, Rolls-Royce Cullinan...), choisir ses dates et ses options, voir **le prix total en direct**, créer un compte, réserver et retrouver ses réservations dans « Mes réservations ».

Le problème qu'il résout

- Les prix sont souvent flous et des frais apparaissent à la fin.
- Réserver prend du temps : appels, e-mails, longs formulaires.

- Il n'y a aucun endroit simple pour retrouver ou annuler sa réservation.

La solution proposée

- Un **catalogue filtrable** (catégorie, carburant, boîte, places, prix) avec photos.
- Un **prix calculé en direct** : remises automatiques de 5 %, 10 % et 15 % selon la durée, 5 options à la journée.
- Des **comptes utilisateurs** sécurisés (Supabase Auth) et des réservations enregistrées en base de données, protégées par une règle de sécurité (RLS).

Public cible

Des clients adultes (25 ans minimum) souhaitant louer une voiture de luxe pour un week-end, un mariage ou un événement.

Type d'application

Application **web** responsive, accessible depuis un navigateur. Pas d'application mobile pour ce MVP.

Pourquoi cette idée plutôt que les autres

- Elle est **plus originale et plus visuelle** que les idées 1, 2 et 4.
- Elle est **plus réaliste dans le temps imparti** que la plateforme de freelances (idée 3) : pas de paiement ni de modération.
- Elle couvre un **périmètre fullstack complet** : pages, calcul de prix, authentification, base de données, sécurité, déploiement.
- Elle reprend l'expérience de MetricFlow (Next.js, Supabase) sans dépendre d'une API externe fragile.
- Elle est **facile à démontrer en direct** devant un public.

Objectifs clés (SMART)

1. **Catalogue et prix** : afficher 12 voitures avec photos, filtres et prix total recalculé selon les dates et les options, livré au Stage 4.
2. **Comptes et réservations** : permettre de créer un compte, de réserver et d'annuler une réservation, chaque utilisateur ne voyant que les siennes, livré au Stage 4.
3. **Mise en ligne** : site accessible publiquement sur Vercel avec un parcours complet testé avant la présentation finale.

Scope (in / out)

In-scope :

- Catalogue, filtres, fiche voiture, 6 agences
- Calcul du prix avec options et remises
- Inscription, connexion, déconnexion
- Réservation, liste « Mes réservations », annulation

Out-of-scope :

- Paiement en ligne
- E-mail de confirmation réel
- Gestion des disponibilités (éviter la double réservation)
- Application mobile native

Risques et défis identifiés

Risque	Mitigation
Sécurité des données de réservation	Utiliser Supabase Auth et des règles RLS pour que chacun ne voie que ses réservations
Photos de voitures : droits d'utilisation	Utiliser des photos sous licence libre (Wikimedia Commons) et les créditer
Dépendance à un service gratuit (Supabase peut se mettre en pause)	Vérifier le projet avant les démos et savoir le restaurer
Environnement de travail qui se réinitialise	Tout faire en ligne de commande et garder le code sur GitHub
Temps limité	Prioriser les fonctions In-scope, garder le paiement pour plus tard

Documentation — Synthèse du processus

Équipe : je suis seul(e) sur ce projet ; j'ai défini les rôles (PM, front, back, design, DevOps, QA) pour rester organisé, avec les mentors Holberton comme repère.

Brainstorming : 6 idées explorées, évaluées selon la faisabilité, l'utilité, l'alignement technique, l'évolutivité et l'impact visuel.

MVP retenu : Driveo, un site de location de voitures de luxe avec prix en direct, comptes utilisateurs et réservations sécurisées.

STAGE 2

Planification du projet

Project Charter — High-Level Plan

Driveo est un site de location de voitures de luxe : le client choisit une voiture, voit le prix total en direct, crée un compte et réserve en quelques clics. Ce document présente le plan global du projet, phase par phase, du lancement jusqu'à la clôture.

Objectifs du projet

- Livrer un site complet et en ligne, avec un parcours client de bout en bout (découvrir, choisir, personnaliser, se connecter, réserver et suivre).
- Garantir la sécurité des données : chaque utilisateur ne voit que ses réservations.
- Pouvoir présenter et démontrer le site en direct.

Timeline / Phases du projet

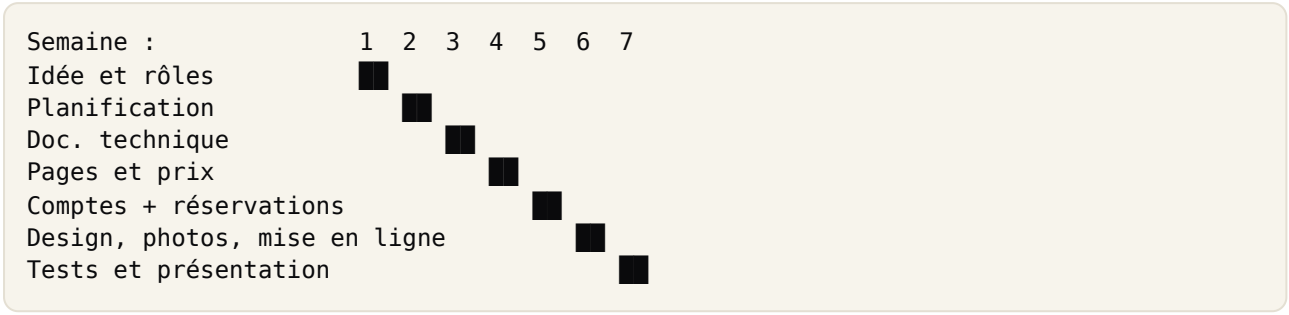
Phase	Période	Objectif	Livrable principal
Stage 1 : Idea Development	Semaine 1	Définir mon rôle, brainstormer, choisir le MVP	Stage 1 Report
Stage 2 : Project Planning	Semaine 2	Définir le plan global du projet	Ce document (Project Charter)
Stage 3 : Technical Documentation	Semaine 3	Concevoir l'architecture, la base de données, les user stories	Documentation technique
Stage 4 : MVP Development	Semaines 4 à 6	Développer les fonctionnalités du MVP	Site fonctionnel en ligne
Stage 5 : Project Closure	Semaine 7	Tester, corriger, préparer la présentation	Présentation finale et démo

Jalons clés

- **Semaine 1** : rôles définis, idée Driveo validée.
- **Semaine 2** : plan global validé, priorités fixées.
- **Semaine 3** : user stories priorisées (MoSCoW), architecture, schéma de base de données.
- **Semaine 4** : pages principales (accueil, catalogue, fiche voiture) et calcul des prix.
- **Semaine 5** : comptes utilisateurs (Supabase Auth) et réservations en base de données avec RLS.
- **Semaine 6** : design noir et or, photos de voitures, mise en ligne sur Vercel.

- **Semaine 7** : tests du parcours complet, corrections, présentation PowerPoint et démo.

Répartition simplifiée (Gantt textuel)



Ressources et outils

Besoin	Outil
Code	Next.js, React, TypeScript, Tailwind CSS
Comptes et base de données	Supabase (Auth, Postgres, RLS)
Versionnement	Git et GitHub
Mise en ligne	Vercel CLI
Photos	Wikimedia Commons (licences libres)

Risques de planification

Risque	Plan B
Retard sur une phase	Réduire le scope (fonctions « Could Have ») plutôt que le délai final
Projet Supabase gratuit en pause	Le restaurer depuis le tableau de bord avant chaque démo
Environnement de travail réinitialisé	Garder tout le code sur GitHub et refaire l'installation en ligne de commande

Points de vigilance pour la suite

- Faire un point de 15 minutes en début de chaque session et un bilan hebdomadaire.
- Terminer le Stage 3 (documentation technique) avant de commencer le développement, pour éviter les allers-retours.
- Tester sur le vrai site en ligne, pas seulement en local.

STAGE 3

Documentation technique

1. User Stories and Mockups

User Stories (priorisées avec MoSCoW)

Must Have

- En tant que client, je veux parcourir un catalogue de voitures avec photos et prix, afin de choisir celle qui me convient.
- En tant que client, je veux filtrer les voitures (catégorie, carburant, boîte, places, prix max), afin de trouver rapidement.
- En tant que client, je veux choisir mes dates et mes options et voir le prix total, afin de savoir ce que je vais payer.
- En tant que client, je veux créer un compte et me connecter de façon sécurisée, afin de réserver.
- En tant que client, je veux réserver une voiture et retrouver ma réservation dans « Mes réservations », afin de la suivre.

Should Have

- En tant que client, je veux annuler une réservation, afin de changer mes plans.
- En tant que client, je veux que le formulaire se pré-remplisse avec mon compte, afin de gagner du temps.
- En tant que client, je veux des remises automatiques pour les longues locations.

Could Have

- En tant que client, je veux recevoir un e-mail de confirmation, afin de garder une trace.
- En tant que client, je veux voir les disponibilités réelles d'une voiture, afin d'éviter les conflits.

Won't Have (hors scope pour ce MVP)

- Paiement en ligne.
- Application mobile native.
- Espace administrateur pour les agences.

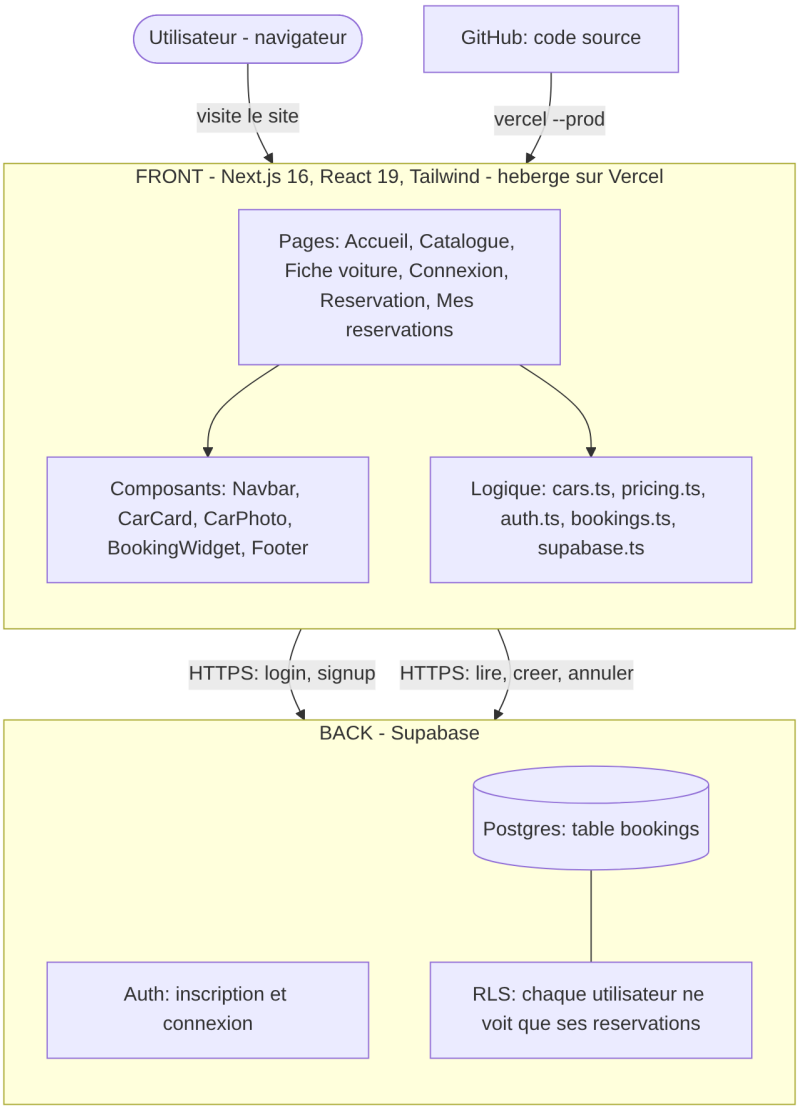
Mockups (écrans principaux)

1. **Accueil** : photo de voiture, recherche (agence, dates), catégories, avis, FAQ.
2. **Catalogue** : filtres à gauche, cartes de voitures avec photo, prix par jour et note.
3. **Fiche voiture** : photo, description, équipements, widget de réservation (dates, options, prix total).

4. **Connexion / inscription** : formulaire e-mail et mot de passe avec photo de l'Urus Performante.
5. **Réservation** : récapitulatif ligne par ligne et confirmation.
6. **Mes réservations** : liste des locations avec bouton d'annulation.

---

2. System Architecture



Flux de données

1. Le client interagit avec le front-end (Next.js).
2. Les voitures et le calcul des prix sont gérés dans le code du front (`cars.ts` , `pricing.ts`).
3. Pour la connexion et l'inscription, le front appelle Supabase Auth via supabase-js.
4. Pour réserver, lire ou annuler, le front appelle la table `bookings` de Supabase.
5. La règle RLS vérifie que l'utilisateur connecté ne touche que ses propres réservations.

---

3. Components, Classes, and Database Design

Schéma de base de données (PostgreSQL via Supabase)

Table `auth.users` (gérée par Supabase Auth)

Champ	Type	Description
id	uuid (PK)	Identifiant de l'utilisateur
email	text	E-mail de connexion
raw_user_meta_data	jsonb	Nom complet saisi à l'inscription

Table `bookings`

Champ	Type	Description
id	uuid (PK)	Identifiant de la réservation
user_id	uuid (FK vers auth.users.id)	Propriétaire, rempli automatiquement avec l'utilisateur connecté
ref	text	Numéro de réservation affiché au client
car_id	text	Identifiant de la voiture (défini dans `cars.ts`)
pickup / dropoff	text	Agences de retrait et de retour
start_date / end_date	date	Dates de location
options	text[]	Options choisies (assurance, GPS, etc.)
total	numeric	Prix total en euros
name, email, phone	text	Coordonnées du client
created_at	timestamptz	Date de création

Règle de sécurité (RLS) : `auth.uid() = user_id` pour toutes les opérations, afin que chacun ne voie et ne modifie que ses réservations.

Les voitures et les agences ne sont pas en base : elles sont définies dans le code (`lib/cars.ts`), car elles changent rarement.

Composants front-end principaux

- `Navbar` — navigation, nom de l'utilisateur, connexion et déconnexion.
- `CarCard` et `CarPhoto` — carte de voiture avec photo et dessin de secours.
- `Catalogue` — filtres et tri des voitures.

- `BookingWidget` — dates, options et prix total sur la fiche voiture.
- `Checkout` — formulaire et récapitulatif de réservation.
- `LoginPage` — inscription et connexion.
- `MesReservations` — liste et annulation des réservations.

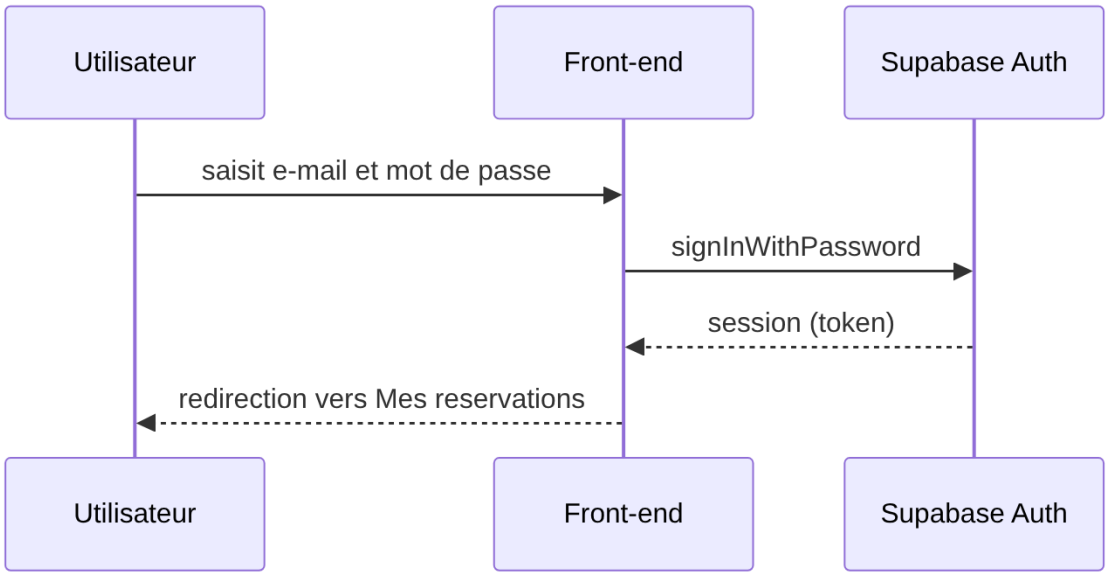
Fichiers de logique

- `cars.ts` — les 12 voitures et les 6 agences.
- `pricing.ts` — prix, remises (5 %, 10 %, 15 %) et options.
- `auth.ts` — hook `useUser` pour l'utilisateur connecté.
- `bookings.ts` — lire, créer et annuler une réservation.
- `supabase.ts` — client Supabase.

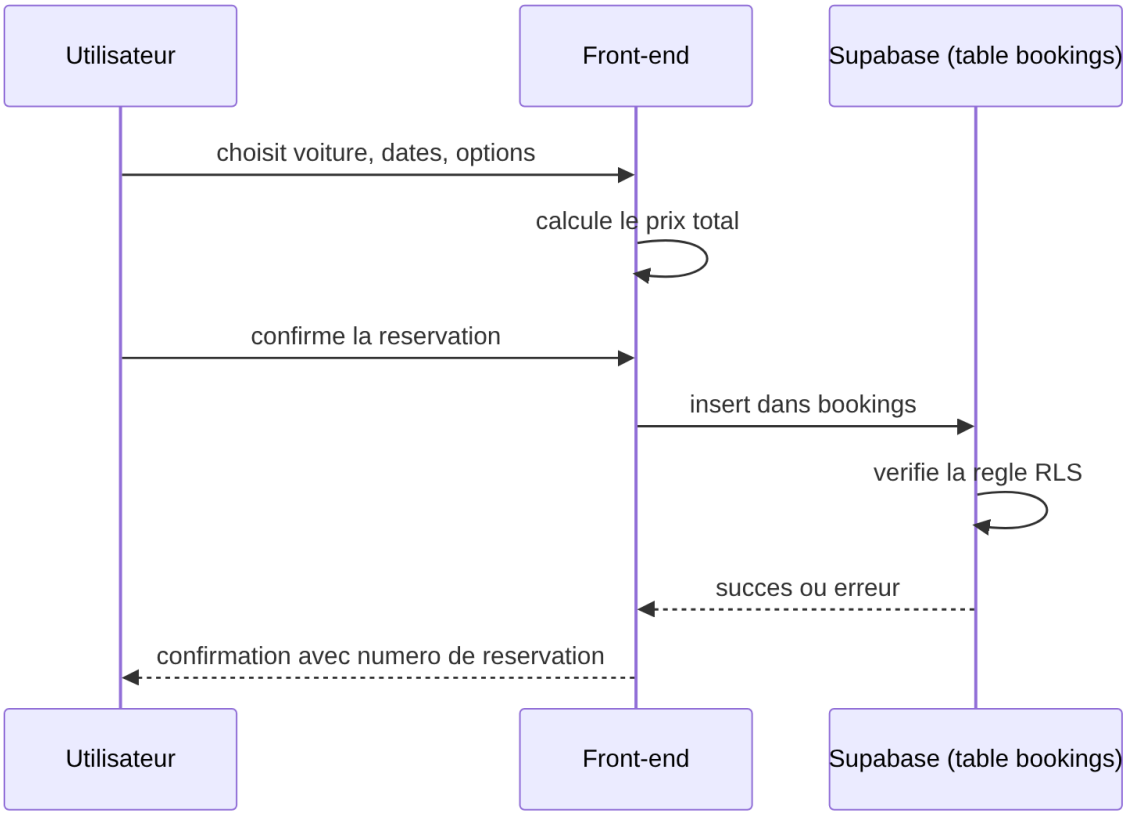
---

4. High-Level Sequence Diagrams

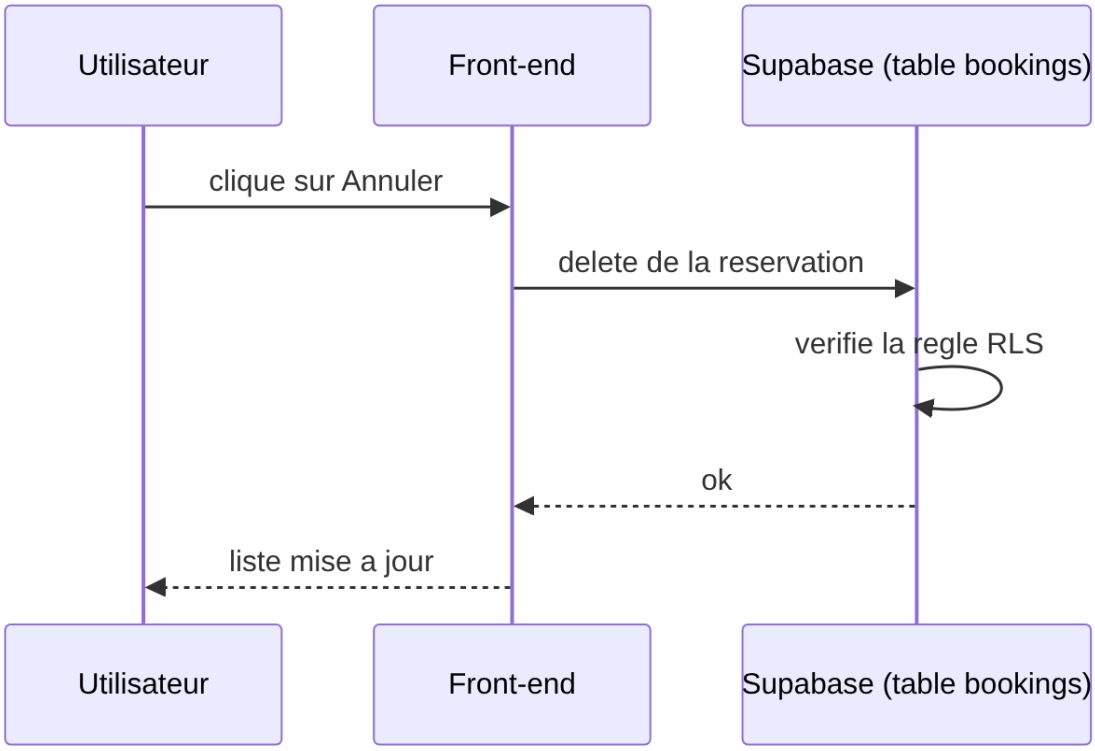
Cas 1 — Connexion



Cas 2 — Réservation



Cas 3 — Annulation



5. API Specifications

Driveo n'a pas de serveur d'API écrit à la main : le front appelle Supabase directement avec supabase-js.

Services externes utilisés

Service	Usage	Pourquoi ce choix
Supabase Auth	Inscription, connexion, session	Sécurisé, gratuit, rapide à mettre en place
Supabase Postgres (API REST automatique)	Lire, créer et supprimer des réservations	Pas de serveur à écrire, sécurité par RLS
Wikimedia Commons	Photos des voitures (licences libres)	Photos libres de droits, avec crédit

Appels utilisés dans le code

Action	Appel supabase-js	Entrée	Sortie
Inscription	`auth.signUp`	e-mail, mot de passe, nom	session ou message à confirmer
Connexion	`auth.signInWithPassword`	e-mail, mot de passe	session
Déconnexion	`auth.signOut`	—	—
Lister mes réservations	`from("bookings").select`	— (session)	liste de réservations
Réserver	`from("bookings").insert`	voiture, dates, options, total, coordonnées	succès ou erreur
Annuler	`from("bookings").delete().eq("ref", ref)`	numéro de réservation	—

6. SCM and QA Plans

Gestion de version (SCM)

- Utilisation de **Git et GitHub** (dépôt `Janice91/metricflow`).

- Branche `main` pour la version en ligne ; commits réguliers avec un message clair (`Driveo luxe : thème noir et or, comptes Supabase, vraies photos`).
- Mise en ligne avec la commande `npx vercel --prod`.

Qualité (QA)

- **Vérification des types** avec TypeScript (`tsc --noEmit`) et **build de production** (`next build`) avant chaque mise en ligne.
- **Tests manuels** du parcours complet : recherche, filtres, fiche voiture, inscription, connexion, réservation, « Mes réservations », annulation.
- **Tests sur le site en ligne** après chaque déploiement, pas seulement en local.
- **Vérification de la sécurité** : un utilisateur ne doit pas voir les réservations d'un autre (règle RLS).

---

7. Technical Justifications

Choix technique	Justification
Next.js 16	Framework React complet : pages, routage et mise en ligne simples
React 19	Composants réutilisables, affichage qui se met à jour tout seul
TypeScript	JavaScript typé, moins d'erreurs avant l'exécution
Tailwind CSS	Design rapide et cohérent (thème noir et or)
Supabase	Comptes, base Postgres et sécurité RLS sans écrire de serveur
Vercel	Mise en ligne en une commande, adaptée à Next.js
GitHub	Versionnement et partage du code avec les pairs
Wikimedia Commons	Photos sous licence libre, créditées sur le site